



PROOF OF AI

Native Execution Proofs in the Hanzo Engine

Zach Kelling

Hanzo AI

June 2026

Abstract

A decentralized AI network must pay operators for work they actually performed. A digital signature proves *authorship* of a message; a plurality vote proves *agreement* among reporters; neither proves *computation* — that the claimed model was run on the claimed input to produce the claimed output. The cheapest robbery of such a network is therefore a one-bit modal guess that joins the quorum and is minted alongside honest work. We present Proof of AI (PoAI): the proof is not a separate artifact bolted onto inference, it is emitted *as a side-effect of execution by the Hanzo engine itself*. The engine is the native inference, training, and embedding execution layer; because $\sim 95\%$ of a forward pass is matrix multiplication $C = A \cdot B$, each layer can be verified in $O(n^2)$ instead of recomputed in $O(n^3)$ using Freivalds’ algorithm over a prime field $\mathbb{F}_p$, $p = 2^{61} - 1$, applied to the *exact int8* integer accumulator. Integer arithmetic is bit-exact across CPU, GPU, and backends, so the check has *zero* false-reject — the load-bearing determinism on which the whole scheme rests. We describe the four proofs that compose over this one primitive (Proof of Embedding, Inference, Training, and Compute), the streaming activation-trace Merkle Mountain Range that lets an operator commit now and be challenged later, the **reportData** binding that ties a proof to a specific model, prompt, and output, and the by-tier treatment of floating point that avoids the unsoundness of naive tolerance-based checking. We state honestly what is proven and tested today and what remains the next slice: the verifier core and determinism prerequisites ship and are tested; the chain enforces no-proof-no-mint; end-to-end proof *emission* from a running model is designed and merged but not yet live.

1 Introduction: what a proof of AI must prove

Consider a network where operators run AI models and are minted tokens for the work. Three properties are easy to confuse, and only one of them is the thing that must be secured.

- A **signature** proves *authorship*: this party committed to this message. It says nothing about whether any computation occurred.
- A **plurality** (a quorum, a majority vote over reported outputs) proves *agreement*: m reporters submitted the same answer. It is only as honest as the reporters, and it rewards consensus, not work.
- A **proof of computation** proves the load-bearing fact: this operator ran *this* model on *this* input and obtained *this* output.

The gap between the first two and the third is an economic vulnerability, not a theoretical nicety. Suppose minting requires only a signed output that matches the quorum. The cheapest attack is then a ~ 1 -bit *modal guess*: an operator that never loads the model, predicts the most likely token (or copies a neighbor), signs it, joins the plurality, and is paid. No matmul is performed; the marginal cost of the “work” is a coin flip. Any network that settles on signatures-plus-agreement is robbed by exactly this strategy, and the robbery is indistinguishable from honest participation at the level of signatures and votes.

PoAI closes the gap by making the proof a *property of execution* rather than a property of reporting. The Hanzo engine is the native execution layer for inference, training, and embedding across the zen model family. When it runs a model it can, at near-zero overhead, emit a commitment to the computation it performed; a verifier later challenges a small random part of that commitment and checks it for asymptotically less than the cost of redoing the work. A modal-guess operator has no committed activations to open and is caught immediately. This paper grounds that claim in the shipped engine code (**hanzo-engine/src/poi.rs**) and its canonical chain-side mirror (**luxfi/crypto/poi**), folds in the product-of-experts settlement material from the earlier PoAI fragment, and is explicit about what is live versus designed.

Contributions.

- The framing above: signature $\neq$ plurality $\neq$ computation, and the modal-guess attack that the distinction exposes (§1).
- The Freivalds core over the exact `int8` accumulator in $\mathbb{F}_p$, with zero false-reject, grounded line-for-line in `poi.rs` (§2).
- The four proofs (PoE/PoI/PoT/PoC) that compose over the single `matmul` primitive via a `workloadType` discriminant (§3).
- The determinism prerequisites that make exact verification possible at all (§4), the commit-now/verify-later activation-trace MMR and interactive bisection (§5), and the `reportData` binding (§6).
- The by-tier floating-point treatment that avoids tolerance-corridor unsoundness (§7) and the architecture-family coverage table for the whole zen zoo (§8).
- An honest implementation-status section (§10).

2 The Freivalds core: verifying a forward pass

A transformer forward pass is dominated by matrix multiplication: the attention projections, the MLP/expert GEMMs, and the output head are all of the form $C = A \cdot B$, and together they are $\sim 95\%$ of the floating (here, integer) operations. This is the lever. To *recompute* a layer $A \in \mathbb{Z}^{t \times k}$, $B \in \mathbb{Z}^{k \times n}$, giving $C \in \mathbb{Z}^{t \times n}$, costs $O(tkn)$ — the same work the prover did. To *verify* it, we do not need to redo it.

Theorem 1 (Freivalds, 1977, over $\mathbb{F}_p$). *Let p be prime, A, B, C integer matrices of conforming shape reduced into $\mathbb{F}_p$, and let $r \in \mathbb{F}_p^n$ be drawn uniformly at random. If $C = A \cdot B$ then $A(Br) = Cr$ always. If $C \neq A \cdot B$ then*

$$\Pr_r[A(Br) = Cr] \leq \frac{1}{p}.$$

Computing Br , then $A(Br)$, and Cr costs $O(tk + kn + tn) = O(n^2)$ matrix-vector work, one order below the $O(n^3)$ of the multiply.

Proof sketch. Let $D = C - A \cdot B \neq 0$ over $\mathbb{F}_p$, so some row $D_i \neq 0$. The event $Dr = 0$ forces $\langle D_i, r \rangle = 0$, a single non-trivial linear constraint on r ; its solution set is a hyperplane containing a $1/p$ fraction of $\mathbb{F}_p^n$. An honest C gives $D = 0$ and passes for every r , which is why there is no false-reject. $\square$

Running k independent challenge vectors multiplies the per-vector soundness error, giving a tampered C a survival probability of at most $(1/p)^k$.

Why integers, and why $p = 2^{61} - 1$. The engine’s `int8` quantized path multiplies signed 8-bit operands into a 32-bit accumulator (`i8 · i8 → i32`, the `f8q8` path in `hanzo-quant`). That accumulator is an *exact integer*; it is identical on every backend that respects the reduction, because integer addition and multiplication carry no rounding. Verifying over $\mathbb{F}_p$ with the Mersenne prime $p = 2^{61} - 1$ keeps every intermediate inside 61 bits, so a single 128-bit multiply (`fmul`) and a conditional subtract (`fadd`) suffice and never overflow. The result is the property the rest of PoAI depends on: **zero false-reject across CPU/GPU/backends**. Soundness error is $1/p \approx 2^{-61}$ per challenge vector; $k = 2$ vectors give $\leq 2^{-122}$.

The shipped API. The core lives in `hanzo-engine/src/poi.rs`: dependency-free, backend-agnostic, unit-tested in isolation. The surface is small and is mirrored byte-for-byte by the Go verifier the chain consumes (`luxfi/crypto/poi`, with `NewMat/Verify/VerifyMulti/DeriveChallenges` and the same $p = (1 \ll 61) - 1$).

```
pub const P: u64 = (1u64 << 61) - 1;          // Mersenne prime field modulus

pub struct Mat { rows, cols, data: Vec<i64> } // exact i32/i64 accumulators

// Verify C == A*B for ONE challenge r: A*(B*r) == C*r over F_p, 0(tk+kn+tn).
pub fn freivalds_verify(a, b, c: &Mat, r: &[u64]) -> bool;

// Verify against k challenges; honest C passes ALL, tampered C fails w.p. >= 1-(1/p)^k.
pub fn freivalds_verify_multi(a, b, c: &Mat, challenges: &[Vec<u64>]) -> bool;

// Derive k challenge vectors of length n from a seed (prod: keccak(beacon || ...)).
pub fn derive_challenges(seed: u64, n: usize, k: usize) -> Vec<Vec<u64>>;
```

The reduction `to_field` uses `rem_euclid` so signed `int8` weights and activations map correctly into $[0, p)$. The module ships eight unit tests covering an honest 2×2 product, a single tampered entry being caught, negative-entry reduction, deterministic challenge derivation, a fail-closed shape mismatch, and a $16 \times 24 \cdot 24 \times 16$ product where flipping one deep entry off-by-one is still caught — the sparse-cheat case that motivates challenging every layer.

The challenge must come after the commitment. Soundness in Theorem 1 assumes r is unpredictable to the prover. If the prover knew r in advance it could craft A, B, C with $Dr = 0$ yet $C \neq A \cdot B$. The production derivation therefore draws the seed from an on-chain beacon *after* the prover has committed (§5); `derive_challenges` is the reproducible expansion both sides apply to that seed. The unit tests use a `splitmix64` stream for the same reproducibility — identical soundness, only the unpredictability source differs.

3 Four proofs over one primitive

PoAI is not four mechanisms; it is one `matmul`-verification primitive applied to four `workloadTypes`. The operands (A, B, C) and the I/O commitment differ by workload, but the verifier, the field, and the soundness argument are shared.

Proof of Embedding (PoE).

The embedding tower (and any ViT/audio encoder feeding it) is a stack of GEMMs producing a vector. Freivalds verifies the projection layers; the input commitment fixes what was embedded. Settlement uses the embedding as the unit of paid work.

Proof of Inference (PoI).

A decode step is the attention and MLP/expert GEMMs plus the output head. The transcript commits the per-layer operands; a challenged layer is opened and Freivalds-checked. This is the canonical case and the one the modal-guess attacker cannot fake.

Proof of Training (PoT).

A training step is a forward pass plus a backward pass; the backward pass is again `matmuls` (the transposed GEMMs of the gradient). The same primitive verifies that a claimed gradient update corresponds to real backpropagation, not a fabricated delta.

Proof of Compute (PoC).

The general case: any committed GEMM workload (a kernel an operator claims to have run for hire) is verifiable by the identical $A(Br) = Cr$ check. PoE/PoI/PoT are PoC specialized by what the operands mean and what I/O is bound.

This is deliberate orthogonal design: one primitive at one slot, four payoffs. A network that wants to pay for embeddings, inference, training, or raw compute does not need four verifiers — it needs one, parameterized by `workloadType`.

Settlement: closed-form expert weights from PoAI (folded from the prior fragment).

Once work is *verified*, it can be *weighted*. For each expert (operator) m , let $q_m \in [0, 1]$ be the Bayesian reliability estimated from its historical attestations. Under a product-of-experts combination the optimal mixing weight is closed-form,

$$\eta_m \propto \frac{q_m}{1 - q_m}, \quad (1)$$

i.e. precision-weighted aggregation, with no manual tuning. At decode time this is the product-of-experts rule

$$\log \pi_*(y | x) = \eta_0 \log \pi_0(y | x) + \sum_{i=1}^K \eta_i \log \phi_i(y | x) - \log Z, \quad (2)$$

where $\pi_0 = p_\theta$ is the base model, $\{\phi_i\}$ are expert factors, and Z is the per-token partition function. PoAI supplies the reliabilities q_m that make Eq. (1) trustworthy: a weight derived from *verified* work, not from self-reported agreement.

4 Determinism is the load-bearing property

Exact Freivalds needs an exactly reproducible C . If two honest runs of the same model on the same input could produce different outputs, there would be no canonical C to commit to, and an honest prover could be false-rejected. Exact verification is therefore *impossible* without bit-level determinism. Two sources of nondeterminism are silently present in ordinary inference, and both are closed in the shipped engine (`hanzoai/engine` v1.2.5).

1. **Argmax ties.** Greedy decode takes $\arg \max$ over logits; when two logits are equal, the winner is implementation-defined (it depends on reduction order on the device). The engine pins a **lowest-id tie-break**: among tied maxima the smallest token index wins, on every backend.
2. **MoE expert selection.** A Mixture-of-Experts router picks the top- k experts by score; ties and unstable sort order change *which experts run*, changing C entirely. The engine pins a **canonical top-k**: sort by score descending, breaking ties by expert index ascending (the `deterministic_topk_indices` selection).

Remark 1. *Determinism is not a performance feature here; it is the precondition for soundness. Only because the `int8` accumulator and these two tie-breaks are reproducible can the verifier assert that an honest operator’s C matches bit-for-bit, and hence that any Freivalds failure is real cheating rather than hardware drift. Proof-bearing runs additionally set `set_moe_proof_deterministic(true)` and pin temperature-0 greedy decode.*

5 Commit now, verify later

A forward pass over a large model at hundreds of tokens produces terabytes of activations; retaining them is infeasible, and forcing synchronous verification would serialize the network. PoAI instead has the operator *commit* to its activations cheaply during execution and be *challenged* later on a small random part — the optimistic bargain: cheap in the happy path, partial re-execution only under challenge.

ActivationTraceMMR. A streaming **Merkle Mountain Range** accumulates one leaf per (layer, token, tap) in execution order and keeps only $O(\log N)$ peak hashes — a frontier of roughly 768 bytes. Activations are hashed as the kernel produces them and then dropped. The hash is **keccak**, with leaf/node domain separation:

$$\begin{aligned} \text{leaf} &= \text{keccak}(\text{DOMAIN_POLACT} \parallel \text{layer} \parallel \text{token} \parallel \text{tap} \parallel \text{dtype} \parallel \text{shape} \parallel \text{tensor bytes}), \\ \text{node} &= \text{keccak}(\text{DOMAIN_POLNODE} \parallel \text{left} \parallel \text{right}), \end{aligned}$$

where $\text{tap} \in \{\text{matmul_in}, \text{matmul_out}, \text{block_out}\}$ — exactly the operands a Freivalds check (or a non-matmul recompute) will open, no more. Lone nodes on odd levels use **RFC-6962 promotion**, *not* Bitcoin duplicate-last: the duplicate-last construction is the CVE-2012-2459 malleability (a forged tree can collide a root), so it is deliberately avoided. The MMR closes by binding the leaf count, $\text{activationTraceRoot} = \text{keccak}(\text{DOMAIN_POI} \parallel n_{\text{leaves}} \parallel \text{mmr root})$, so a subtree cannot be reinterpreted as the whole tree.

Zero overhead when off. The commitment is threaded as an `Option<mut ProofTranscript>` through the forward context, not a boolean checked in hot loops. When the option is `None`, each commit site is a single not-taken branch — no hashing, no allocation, no tensor copy. When it is `Some`, the per-matmul hook commits the int8 activation view and the i32 accumulator by borrow (no copy); the weight B is committed once at load into the weights root, never re-hashed per token.

Challenge, open, bisect. A verifier derives a layer index ℓ and challenge r from a beacon *after* commitment, opens that layer’s A (**matmul_in**), B (a reference under the weights root), and C (**matmul_out**) with $O(\log N)$ MMR inclusion proofs, and runs *windowed* Freivalds over a random token sub-range to keep the opened payload small. The verifier recomputes the leaf hashes, checks the inclusions against the committed roots, re-derives r , and checks $A(Br) = Cr$ — total cost $O(tk + kn + tn + \log N)$, about $\min(t, k, n) \times$ cheaper than recomputing the layer. A mismatch slashes the operator. To localize a *sparse* cheat (a single bad layer among many) the protocol runs interactive **bisection** to the first divergent layer in $O(\log L)$ rounds, so soundness is independent of how many layers are sampled.

6 Binding: a proof points at exactly one (model, input, output)

A correct matmul check is worthless if the attestation can be replayed, spliced across tasks, or pointed at a different model. PoAI binds the proof with two hash chains whose constants match the chain side byte-for-byte. The challenge is derived from immutable task context:

$$\begin{aligned} \text{challenge} &= \text{keccak}(\text{DOMAIN_CHALLENGE} \parallel \text{taskId} \parallel \text{intentID} \parallel \text{modelSpecHash} \\ &\quad \parallel \text{promptHash} \parallel \text{openBlockHash} \parallel \text{operator}), \end{aligned}$$

and the attested result commits to that challenge:

$$\text{reportData} = \text{keccak}(\text{DOMAIN_REPORT} \parallel \text{challenge} \parallel \text{modelSpecHash} \parallel \text{promptHash} \parallel \text{outputHash} \parallel \text{runtimeMeasurement}).$$

The chain consumes `reportData`; because it folds in the challenge (which already folds in `taskId`, `intentID`, and `openBlockHash`), a transcript is task-scoped and cannot be reused.

modelSpecHash is measured, not labelled. The model identity is a keccak Merkle root over the *quantized weight bytes the engine actually loaded* (the per-layer `QuantizedSerde` serialization, with the quant-type discriminant in each leaf). It is not a name string. Re-quantizing the same weights to a different scheme yields a different root; a genuinely different model yields a different root. Combined with a `runtimeMeasurement` that folds in the kernel build id, reduction mode, and sampler/seed, the decoding regime becomes part of identity: the same weights decoded at temperature 2.0, or under a different reduction order, are rejected. The full transcript root chains these together,

$$\text{transcriptRoot} = \text{keccak}(\text{DOMAIN_POI} \parallel \text{modelID} \parallel \text{inputCommitment} \parallel \text{quantizationSpec} \parallel \text{kernelVersion} \parallel \text{activationTraceRoot} \parallel \text{outputCommitment}),$$

so the binding runs `challenge` $\rightarrow$ `model` $\rightarrow$ `input` $\rightarrow$ `output` and an attestation cannot be detached from any link in the chain.

7 Floating point: by tier, not by tolerance

The natural temptation is to run Freivalds directly on floating-point GEMMs with a tolerance: accept if $\|A(Br) - Cr\| < \varepsilon$. This is both **unsound** and **false-rejecting**, and PoAI rejects it.

- *Unsound*: a tolerance corridor of width ε is an attacker’s budget. A cheater can perturb C anywhere inside the corridor — a non-trivial space of fabricated outputs — and pass. The clean $1/p$ soundness of Theorem 1 holds only in an exact field.
- *False-rejecting*: floating-point reduction order is not fixed across hardware, so two honest runs differ in the low bits. Any ε small enough to be sound is small enough to reject honest operators on a different GPU.

PoAI therefore selects the verification regime by numeric tier:

Table 1: Verification regime by numeric tier.

Tier	Mechanism	Soundness basis
<code>int8</code> / quantized	exact Freivalds over $\mathbb{F}_p$	software-only, zero false-reject
<code>fp16</code> / <code>bf16</code> / <code>fp8</code>	deterministic-fp (RepOps): pin reduction order	bit-reproducible by pinned reduction; or TEE/CC attestation
<code>small</code> / <code>niche</code>	zkML circuit	succinct cryptographic proof

For the quantized path the check is exact and software-only — no special hardware, the common case for the deployed zen models. For native low-precision floating point, the fix is not a tolerance

but **determinism**: pin the reduction order (RepOps / deterministic-fp) so the GEMM is bit-reproducible, then the exact equality check applies again; where that is impractical, fall back to a trusted-execution / confidential-compute attestation of the kernel. For small or unusual workloads a zkML circuit gives a succinct proof directly.

Remark 2. A floating-point value is an integer — a sign bit, an exponent, and a mantissa. A GEMM executed in a pinned reduction order is therefore an exact, reproducible computation over those integers; “deterministic-fp” is not an approximation but a commitment to a specific, replayable order of exact operations. The unsoundness of naive fp-Freivalds comes entirely from unpinned order, not from floating point being inherently fuzzy.

8 Coverage across the zen architecture families

The matmul primitive plus two extensions covers the entire zen model zoo. The two extensions are a *routing proof* (for sparse/MoE models) and *sequential state-chaining* (for recurrent and multi-step models); every other family is base Freivalds plus a deterministic input commitment.

Table 2: Architecture-family coverage. “Beyond base” marks the two extensions over plain dense-matmul Freivalds.

Family	How PoAI verifies it	Beyond base
Dense transformer	Freivalds on every layer’s GEMMs; activation-trace MMR.	—
Mixture-of-Experts	Commit router logits + selected expert indices per token; verify the <i>canonical</i> top- k chose the same experts, then Freivalds the selected experts’ GEMMs only (sparse).	routing proof
Linear-recurrent / SSM / Gated DeltaNet	Per-step Freivalds, with consecutive states bound in the commitment (step t ’s output is step $t+1$ ’s input).	state-chaining
Multimodal (ViT + audio)	Vision/audio towers are GEMMs → Freivalds; commit the preprocessed input by hash; pin preprocessing (resize/normalize; FFT→mel is commit-and-recompute).	input commitment
Diffusion / Transfusion	Multi-step: per-step Freivalds plus binding consecutive latents in the commitment, exactly the recurrent chaining payoff reused.	state-chaining

One primitive, two extensions, full coverage — the recurrent state-chaining mechanism serves both linear-recurrent models and diffusion, so there is no separate diffusion verifier.

9 Ownership: whose data, whose compute

PoAI is the technical substrate for an ownership model. Users own their data and own the AI they run — the proof lets them prove a computation was performed without surrendering control of the model or the inputs to a trusted middleman. Compute is *selectable*: a user may have a trusted DAO node perform the work (convenient, with the proof guaranteeing they were not short-changed) or run a user-operated node themselves (sovereign, with the same proof format settling on the same chain). The verification is identical in both cases; what differs is only who holds the keys to the executing node. The proof, not the operator’s reputation, is what gets minted against.

10 Implementation status

We state plainly what exists, what is enforced, and what is not yet live.

Table 3: Component status. “Proven+tested” means shipped with green tests; “enforced” means the chain rejects non-compliant claims; “designed” means the spec is merged but no running model emits it yet.

Component	Where	State
Go Freivalds verifier (chain-consumed)	luxfi/crypto/poi-v1.19.23	Proven + tested (canonical mirror)
Rust Freivalds core (prover mirror)	hanzo-engine/srE/prover-v1.2.5	Proven + tested (8 tests green)
Determinism prerequisites	hanzoai/engine-v1.2.5	Shipped (lowest-id argmax; canonical MoE top- k)
No-proof-no-mint enforcement	luxdao/contract-v2.1.0	Enforced (230/230 forge tests)
Soundness companion	zooai/proofs	Written (tolerance-vs-exact analysis)
Prover-side core in node binaries	hanzod 1.1.20 (embeds engine)	Available via path-dep on hanzo-engine v1.2.5
Verifier in Go node binaries	luxd, zood	crypto/poi v1.19.23 builds in-module (zood graph-sync pending)
End-to-end proof <i>emission</i> from a running zen model	forward-pass commitment slice	Designed + merged, NOT yet live

To be unambiguous: the verification core is **proven and tested** in both Rust (the engine prover mirror) and Go (the chain verifier), the determinism prerequisites are shipped, and the chain **enforces** that no mint occurs without a proof. What is *not* yet live is the end-to-end emission: threading the `ActivationTraceMMR` and the per-layer commit hooks through the engine’s forward

pass so that a real zen model (e.g. an `int8` MoE) produces a transcript that a challenger opens and verifies. That slice is designed and its wire constants are merged, but no running model emits a proof today. The first true “live” is a green end-to-end test in which an honest `int8` zen inference emits a transcript, a beacon-derived layer is challenged, the open verifies — and a faked, cheap-model, or mis-routed run is caught. That is the next slice, not a present claim.

11 Conclusion

A network that pays for AI must verify computation, not authorship or agreement. The decisive observation is that a forward pass is almost entirely matmuls, that a matmul can be *verified* an order of magnitude below the cost of *performing* it, and that on the engine’s exact integer accumulator this verification is bit-exact with zero false-reject. PoAI builds directly on the Hanzo engine: the engine is the native execution layer, and the proof is emitted as a side-effect of that execution — one Freivalds primitive over $\mathbb{F}_p$, specialized into four proofs (embedding, inference, training, compute), made sound by determinism, made asynchronous by a streaming Merkle Mountain Range with interactive bisection, and made unforgeable by a measured-weights binding into `reportData`. The core is proven and tested and the chain already refuses to mint without a proof; emitting that proof end-to-end from a running zen model is the next, designed, slice.

References

- [1] R. Freivalds. Probabilistic machines can use less running time. *IFIP Congress*, 1977.
- [2] Hanzo AI. `hanzo-engine/src/poi.rs` — Proof-of-Inference Freivalds core. `hanzoai/engine v1.2.5`.
- [3] Lux. `crypto/poi` — canonical Go Freivalds verifier (`NewMat`, `Verify`, `VerifyMulti`, `DeriveChallenges`). `luxfi/crypto v1.19.23`.
- [4] Hanzo AI. Proof-of-Inference: forward-pass commitment design. `hanzo-engine/docs/proof-of-inference/`, 2026.
- [5] Zoo. Proof-of-Inference soundness: exact field versus tolerance corridor. `zooai/proofs`, 2026.
- [6] Lux DAO. `ComputeProofLib` — on-chain `reportData` binding and no-proof-no-mint enforcement. `luxdao/contracts v2.1.0`.
- [7] B. Laurie, A. Langley, E. Kasper. Certificate Transparency. RFC 6962, 2013. (Lone-node promotion; avoids CVE-2012-2459 duplicate-last malleability.)
- [8] Hanzo AI. Hanzo Consensus AI: Consensus Mechanisms for Distributed AI Training and Inference. `hanzoai/papers`, 2026.